Optimal transfer-learning workflow

1. **Define the task and constraints**
    - Exact input and output
    - Number of classes
    - Target platform: server, mobile, or both
    - Primary metric, such as top-1 accuracy
    - Secondary metrics: top-2 accuracy, loss, latency and model size
    - Hardware, memory and inference-time limits
    - Acceptable confidence and error behavior
2. **Validate the data**
    - Confirm every image loads
    - Validate labels and bounding boxes
    - Repair or remove invalid samples
    - Inspect class distribution
    - Detect duplicates and data leakage
    - Create fixed training, validation and test splits
    - Never tune parameters using the final test set
3. **Build the input pipeline**
    - Crop the relevant object
    - Make input size configurable
    - Resize or pad without unwanted distortion
    - Normalize using the pretrained model's expected mean and standard deviation
    - Add only safe, basic augmentation initially
    - Build datasets, samplers and data loaders
    - Decide how to handle class imbalance
    - Visually inspect transformed samples
    - Verify final tensor shapes

4. **Create reproducible experiment infrastructure**
    - Set random seeds
    - Save model and optimizer checkpoints
    - Support resuming interrupted training
    - Record every experiment's configuration
    - Use unique experiment and checkpoint names
    - Track training loss, validation loss, accuracy, learning rates and epoch
    - Save best-loss, best-accuracy and latest checkpoints separately
    - Change one main variable at a time
5. **Choose a pretrained backbone**
    - Select a model appropriate for the deployment target
    - Load pretrained weights
    - Replace its output layer for the new classes
    - Verify input and output tensor shapes
    - Record parameter count, model size and expected computation
    - Optionally select one alternative backbone for later comparison
6. **Establish a standard frozen-backbone baseline**
    - Freeze all feature-extraction layers
    - Train only the classifier head
    - Begin with the original pretrained classifier structure when possible
    - Use a straightforward optimizer and learning rate
    - Train until validation performance stops improving
    - Save best-loss and best-accuracy checkpoints
    - Use this model as the reference later experiments must beat
7. **Compare classifier-head architectures**
    - Compare a few meaningful structures:
        - Linear output head
        - Smaller hidden layer

        - Original pretrained hidden layer
    - Train a fair frozen-backbone baseline for each
    - Compare accuracy, loss, parameter count and model size
    - Select the head before detailed hyperparameter tuning
8. **Choose the input resolution**
    - Compare a small number of realistic resolutions
    - Start every comparison from the same appropriate baseline
    - Keep all other settings fixed
    - Compare:
        - Top-1 accuracy
        - Top-2 accuracy
        - Validation loss
        - Training and inference speed
        - Memory consumption
    - Select the resolution before detailed fine-tuning
    - Preserve a smaller resolution as a mobile fallback if useful
9. **Select the fine-tuning depth**
    - Begin with the best frozen baseline
    - Compare classifier-only training against partial unfreezing
    - Unfreeze the deepest feature module first
    - Compare one, two or more deep modules only when justified
    - Keep early generic feature layers frozen unless deeper unfreezing clearly helps
    - Watch for training loss decreasing while validation loss increases
10. **Choose the BatchNorm policy**
    - Remember that BatchNorm normalizes activations, not weights
    - Distinguish between:
        - Trainable gamma and beta

        - Stored running mean and variance
    - Start with all BatchNorm layers in evaluation mode
    - Compare against adapting BatchNorm only in unfrozen modules
    - Never accidentally update BatchNorm statistics in frozen modules
    - Consider batch size because small batches produce noisy statistics
    - Keep the policy that improves validation performance consistently
11. **Tune optimization**
    - Use separate optimizer parameter groups:
        - Higher learning rate for the classifier
        - Lower learning rate for pretrained features
    - Compare a small number of learning rates
    - Choose an optimizer, such as AdamW or SGD
    - Add an appropriate learning-rate scheduler
    - Set a generous epoch limit with early stopping
    - Choose batch size mainly from memory and training stability
    - Check that every trainable parameter is included in the optimizer
12. **Tune data augmentation**
    - Begin with realistic transformations:
        - Horizontal flipping
        - Small rotations
        - Small translations
        - Small scale changes
        - Mild shear
        - Mild color changes when appropriate
    - Visually inspect augmentation examples
    - Optionally test stronger methods:
        - MixUp
        - CutMix

        - Random Erasing
    - Never apply random augmentation to validation or test data

13. **Tune regularization**
    - Label smoothing
    - Weight decay
    - Dropout
    - Test a few meaningful values rather than every possible combination
    - Compare validation loss, top-1 and top-2 together
    - Prefer the simpler configuration when results are effectively tied
    - Avoid stacking multiple new regularizers in one experiment
14. **Perform error analysis**
    - Generate a confusion matrix
    - Calculate per-class accuracy
    - Inspect the weakest classes
    - Find frequently confused class pairs
    - Examine incorrectly classified images manually
    - Determine whether errors come from:
        - Visually similar classes
        - Incorrect labels
        - Poor crops
        - Low image quality
        - Class imbalance
        - Insufficient model capacity
        - Missing contextual information
15. **Confirm the winner robustly**
    - Re-evaluate promising checkpoints in full precision
    - Compare best-loss and best-accuracy checkpoints
    - Repeat finalists with multiple random seeds when practical

- Compare average results, not only the luckiest run
- Consider the number of additional correct predictions, not just percentages
- Keep a simpler model if additional complexity produces negligible improvement

16. **Evaluate once on the untouched test set**
    - Freeze all architecture and hyperparameter decisions first
    - Run top-1, top-2, loss and per-class evaluation
    - Treat this as the final estimate of generalization
    - Do not continue tuning based on the test result
    - Create a new test set for future development cycles if necessary
17. **Calibrate confidence**
    - Check whether predicted confidence matches actual correctness
    - Calculate calibration metrics
    - Apply temperature scaling if needed
    - Define a low-confidence threshold
    - Decide when to display:
        - One prediction
        - Two possible breeds
        - An “uncertain” result
18. **Export and verify**
    - Export the selected checkpoint to ONNX
    - Store class labels and preprocessing metadata
    - Confirm ONNX outputs match PyTorch outputs
    - Test fixed or dynamic input shapes
    - Benchmark latency and memory
    - Try FP16 or INT8 quantization
    - Re-evaluate accuracy after conversion and quantization
19. **Integrate and monitor**

- Connect detector crops to the classifier
- Ensure production preprocessing exactly matches training
- Test real camera images, not only dataset images
- Measure complete detector-plus-classifier latency
- Test multiple dogs and poor-quality crops
- Log low-confidence and incorrect predictions
- Use real-world failures to plan the next dataset and training cycle

The condensed order is:

task → data → pipeline → infrastructure → backbone → frozen baseline → head architecture → resolution → unfreezing depth → BatchNorm → optimization → augmentation → regularization → error analysis → robust evaluation → test set → calibration → export → deployment.